Quantum Neural Networks

Theory, Architectures, and Applications

1 Background

Classical machine learning has gained huge success in recent years especially deep learning models such as multi-layer perceptrons (MLP), Convolution Neural Networks (CNN), Recurrent Neural Netowrks (RNN), Transformers, and Graph Neural Netowrks (GNN) [1, 2, 3]. These architectures, due to its inductive bias and with development of hardware accelerators, have been applied in many fieds such as image processing, audio-speech processing, language modeling, and fraud detection. However, as the models are getting larger and larger, the computing power required is also growing exponentially. Governments and private companies are spending billions of dollar on building data center to satisfy the compute needs of now.

On the other hand, a new paradigm of computing started to shine on the stage – quantum computing [4].

2 Review of Classical Neural Networks

Before exploring Quantum Neural Networks (QNNs), it is essential to understand their classical counterparts. At their core, neural networks are parameterized mathematical models designed to approximate complex functions from data. Among the various architectures, the Multilayer Perceptron (MLP) is one of the most fundamental and widely studied. Despite its relatively simple structure, the MLP provides the theoretical foundation for understanding the expressive power of neural networks.

2.1 Architecture of Multilayer Perceptrons

An MLP is a feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer, as illustrated in Fig. 1 [5, Chap. 6]. Each layer consists of multiple neurons, which perform affine transformations followed by nonlinear activation functions.

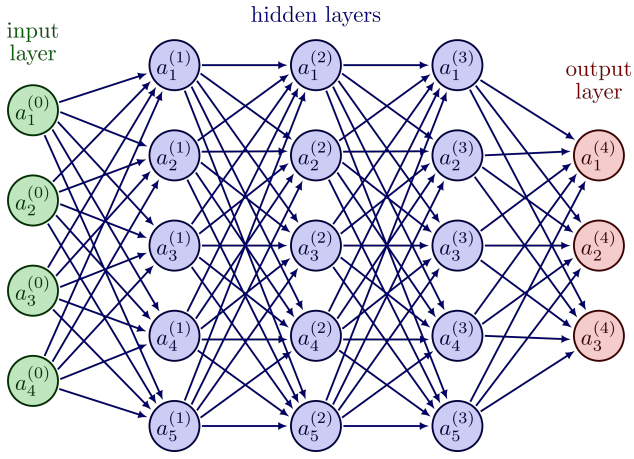


Figure 1: Architecture of a Multilayer Perceptron (MLP), consisting of an input layer, hidden layers, and an output layer.

Mathematically, the output of the l-th hidden layer can be expressed as

$$\mathbf{h}^{(l)} = \sigma\left(W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) \tag{2.1}$$

where $\mathbf{h}^{(0)} = \mathbf{x}$ denotes the input vector, $W^{(l)}$ and $\mathbf{b}^{(l)}$ are the trainable weight matrix and bias vector, respectively, and σ is a nonlinear activation function. Common choices include Sigmoid, Tanh, and Rectified Linear Unit (ReLU).

By composing multiple layers, an MLP constructs a parameterized mapping from the input space to the output space. Let $A^{(l)}(\mathbf{h}) = W^{(l)}\mathbf{h} + \mathbf{b}^{(l)}$ denote the affine transformation at layer l. An MLP with L affine layers and a linear output layer can then be represented as

$$f_\theta(\mathbf{x}) = A^{(L)} \circ \sigma \circ A^{(L-1)} \circ \cdots \circ \sigma \circ A^{(1)}(\mathbf{x}) \tag{2.2}$$

where θ represents all trainable parameters and $\circ$ denotes function composition.

An important observation is that the nonlinear activation function is essential. Without nonlinearities, any composition of affine transformations remains an affine transformation, regardless of the number of layers. Therefore, nonlinear activations are what enable MLPs to represent functions beyond simple linear mappings.

2.2 MLPs as Function Approximators

The fundamental objective of an MLP is to approximate an unknown target function $f^*(\mathbf{x})$ using a parameterized function $f_\theta(\mathbf{x})$.

To understand how this works, consider a single-hidden-layer MLP with a scalar output:

$$f_\theta(\mathbf{x}) = \sum_{j=1}^{m} a_j \sigma(\mathbf{w}_j^T \mathbf{x} + b_j) + c \tag{2.3}$$

where m is the number of hidden neurons, $\mathbf{w}_j$ and b_j determine the transformation applied by the j-th neuron, and a_j controls its contribution to the final output.

This expression reveals a fundamental principle: an MLP approximates a complex function by combining multiple simpler nonlinear functions. Each hidden neuron generates a parameterized basis-like function, and the output layer constructs a weighted combination of these functions.

For example, consider the ReLU activation:

$$\sigma(x) = \max(0, x) \tag{2.4}$$

Each ReLU neuron introduces a piecewise-linear component. By combining sufficiently many such components, an MLP can construct increasingly complex piecewise-linear functions that approximate nonlinear target functions, as illustrated in Fig. 2.

Given a training dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, the network learns its parameters by minimizing a loss function. For example, in regression tasks, the Mean Squared Error (MSE) objective is

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \left(f_\theta(\mathbf{x}_i) - y_i\right)^2 \tag{2.5}$$

In practice, the parameters are typically optimized through gradient-based methods using backpropagation [6].

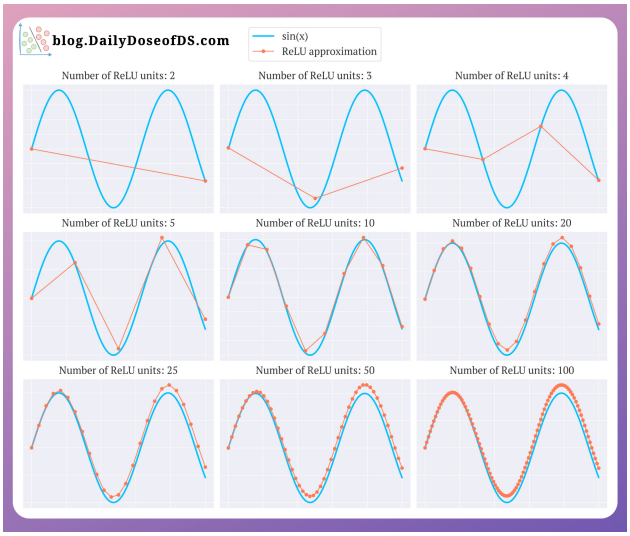


Figure 2: Illustration of function approximation using ReLU-based MLPs. Increasing the number of hidden neurons provides greater flexibility to approximate a nonlinear target function.

2.3 Universal Approximation Theorem

A natural question arises: how complex a function can an MLP approximate?

One of the most important theoretical results in neural network research is the Universal Approximation Theorem (UAT), established through a series of foundational works [7, 8].

In particular, consider a continuous target function $f^* : K \to \mathbb{R}$, where $K \subset \mathbb{R}^d$ is compact. For any approximation tolerance $\epsilon > 0$, there exists a single-hidden-layer ReLU network with sufficiently many hidden neurons such that

$$\boxed{\sup_{\mathbf{x} \in K} |f^*(\mathbf{x}) - f_\theta(\mathbf{x})| < \epsilon} \tag{2.6}$$

In other words, even an MLP with only one hidden layer can approximate any continuous function on a compact domain to arbitrary accuracy, provided that sufficient hidden neurons are available.

This theorem establishes the remarkable expressive potential of neural networks. However, several important distinctions must be made.

First, the theorem guarantees the *existence* of suitable parameters, but does not guarantee that a training algorithm can efficiently discover them. Second, it does not specify how many neurons or parameters are required to achieve a desired approximation accuracy. Finally, approximation capability does not necessarily imply good generalization to unseen data.

Although a single hidden layer is theoretically sufficient for universal approximation, deeper architectures may represent certain functions more efficiently by exploiting their compositional structure [5, 9].

2.4 Beyond MLPs: Toward Quantum Neural Networks

Modern deep learning has extended the basic principles of MLPs into numerous specialized architectures. Convolutional Neural Networks (CNNs) exploit spatial structure, recurrent architectures such as RNNs, LSTMs, and GRUs model sequential dependencies, Transformers leverage attention mechanisms, and Graph Neural Networks (GNNs) incorporate graph structure [5, 2, 3]. Despite their architectural differences, these models share the fundamental objective of learning parameterized functions from data.

This naturally leads to an intriguing question: can quantum computers also serve as function approximators?

While classical MLPs construct functions through compositions of affine transformations and nonlinear activations, variational QNNs construct functions through quantum data encoding, parameterized unitary transformations, and measurements. A typical QNN output can be expressed as [10, 11]

$$f_\theta(\mathbf{x}) = \langle 0|^{\otimes n} U^\dagger(\mathbf{x},\theta)\mathcal{O}U(\mathbf{x},\theta)\,|0\rangle^{\otimes n} \tag{2.7}$$

where $U(\mathbf{x},\theta)$ represents the parameterized quantum circuit and $\mathcal{O}$ denotes a measurement observable.

Thus, both MLPs and QNNs can be viewed as parameterized function approximators, yet they rely on fundamentally different computational mechanisms. This raises several important questions: How do QNNs represent functions? What determines their expressive power? And how does their function approximation capability compare with that of classical MLPs?

These questions motivate the following investigation into the mathematical representation and expressive properties of Quantum Neural Networks.

3 Quantum Neural Networks

3.1 Basics of Quantum Computing

Quantum computing is built upon the principles of quantum mechanics, which provide a fundamentally different computational framework from classical computing. Unlike classical computers, which process information using binary bits, quantum computers manipulate quantum bits, or qubits, through unitary transformations and quantum measurements. This section introduces the essential concepts of quantum computing, including quantum states, quantum gates, tensor products, entanglement, and measurement. These concepts form the mathematical foundation of Quantum Neural Networks (QNNs). The following discussion is based on the standard quantum computing formalism presented in [4, Chaps. 2 and 4].

3.1.1 Qubits and Quantum States

In classical computing, a bit can take one of two possible values, either 0 or 1. In contrast, a qubit can exist in a superposition of two computational basis states, denoted by $|0\rangle$ and $|1\rangle$. The general state of a single qubit is expressed as

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \qquad |\alpha|^2 + |\beta|^2 = 1 \tag{3.1}$$

where $\alpha, \beta \in \mathbb{C}$ are complex probability amplitudes. In vector notation,

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad |\psi\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \tag{3.2}$$

According to the Born rule, the probabilities of measuring $|0\rangle$ and $|1\rangle$ are $|\alpha|^2$ and $|\beta|^2$, respectively. Importantly, the probability amplitudes also contain phase information, which plays a crucial role in quantum interference.

Ignoring an unobservable global phase, any pure single-qubit state can be represented geometrically on the Bloch sphere, as illustrated in Fig. 3:

$$|\psi\rangle = \cos\left(\frac{\vartheta}{2}\right)|0\rangle + e^{i\varphi}\sin\left(\frac{\vartheta}{2}\right)|1\rangle \tag{3.3}$$

where ϑ and φ represent the polar and azimuthal angles, respectively.

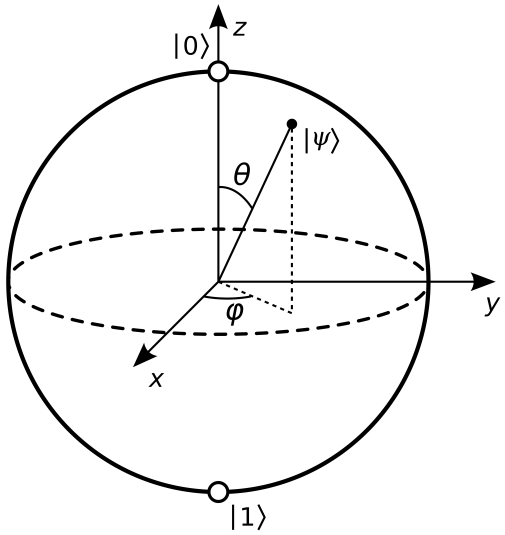


Figure 3: Bloch sphere representation of a single-qubit pure state. The angles ϑ and φ characterize its position on the sphere, up to a global phase.

For a system containing n qubits, the quantum state belongs to a 2^n-dimensional Hilbert space and can be expressed as

$$|\psi\rangle = \sum_{j=0}^{2^n-1} a_j\,|j\rangle\,, \qquad \sum_{j=0}^{2^n-1} |a_j|^2 = 1 \tag{3.4}$$

where $|j\rangle$ denotes the computational basis state corresponding to the binary representation of j.

Throughout this paper, we adopt the *little-endian convention*, in which the computational basis is ordered as $|q_{n-1}\cdots q_1 q_0\rangle$, with q_0 being the least significant bit.

Although the dimension of the quantum state space grows exponentially with the number of qubits, this does not automatically imply an exponential computational advantage, since extracting information from a quantum state is constrained by quantum measurement.

3.1.2 Quantum Gates and Unitary Operations

Quantum states are manipulated through quantum gates, which are mathematically represented by unitary matrices. A quantum gate U transforms an initial state $|\psi\rangle$ into

$$|\psi'\rangle = U|\psi\rangle, \qquad U^\dagger U = I \tag{3.5}$$

where $U^\dagger$ is the conjugate transpose of U. The unitary condition ensures that the normalization of the quantum state is preserved throughout the evolution of an ideal quantum circuit.

Several fundamental single-qubit gates are widely used in quantum computing. For example, the Pauli-X gate acts similarly to a classical NOT gate, while the Hadamard gate transforms computational basis states into equal superpositions:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \tag{3.6}$$

In particular,

$$H|0\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \tag{3.7}$$

Another important class of gates is the rotation gates, which rotate quantum states around different axes of the Bloch sphere. They can be expressed as

$$R_\nu(\theta) = e^{-i\theta\sigma_\nu/2}, \qquad \nu \in \{x, y, z\} \tag{3.8}$$

where σ_ν denotes the corresponding Pauli operator. For example, the rotation around the X-axis is

$$R_x(\theta) = \begin{bmatrix} \cos\frac{\theta}{2} & -i\sin\frac{\theta}{2} \\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{bmatrix} \tag{3.9}$$

Rotation gates are particularly important in QNNs because their rotation angles can encode classical input features or represent trainable model parameters.

When multiple qubits are involved, the combined state space is constructed using the tensor product. If independent single-qubit gates $U_0, U_1, \ldots, U_{n-1}$ act on their respective qubits, the overall operation is represented, under our little-endian convention, as

$$U = U_{n-1} \otimes \cdots \otimes U_1 \otimes U_0 \tag{3.10}$$

The tensor product provides a systematic way to construct multi-qubit operations from individual gates. However, not all multi-qubit operations can be decomposed into tensor products of single-qubit gates. Such operations are essential for generating quantum entanglement.

3.1.3 Multi-Qubit Gates and Entanglement

One of the most fundamental multi-qubit gates is the Controlled-NOT (CNOT) gate, which operates on two qubits: a control qubit and a target qubit. The target is flipped if and only if the control is in state $|1\rangle$. Its action on computational basis states is defined as

$$|c, t\rangle \xrightarrow{\text{CNOT}} |c, t \oplus c\rangle \tag{3.11}$$

where $\oplus$ denotes modulo-2 addition.

For a two-qubit system following the little-endian convention $|q_1 q_0\rangle$, if q_1 is the control and q_0 is the target, the corresponding matrix is

$$\text{CNOT}_{q_1 \to q_0} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{3.12}$$

For example, $\text{CNOT}_{q_1 \to q_0} |10\rangle = |11\rangle$.

Conversely, if q_0 is the control and q_1 is the target, the matrix becomes

$$\text{CNOT}_{q_0 \to q_1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \tag{3.13}$$

Notice that reversing the control and target qubits changes the matrix representation. For instance,

$$\text{CNOT}_{q_0 \to q_1} |01\rangle = |11\rangle \tag{3.14}$$

Beyond conditional bit flipping, the CNOT gate plays a crucial role in creating quantum entanglement. Consider two qubits initially prepared in $|00\rangle$. As illustrated in Fig. 4, we first apply a Hadamard gate to q_0:

$$\begin{aligned} |\psi\rangle &= (I \otimes H) |00\rangle \\ &= \frac{1}{\sqrt{2}} (|00\rangle + |01\rangle) \end{aligned} \tag{3.15}$$

Next, applying a CNOT gate with q_0 as the control and q_1 as the target produces

$$\begin{aligned} |\psi_{\text{Bell}}\rangle &= \text{CNOT}_{q_0 \to q_1} |\psi\rangle \\ &= \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle) \end{aligned} \tag{3.16}$$

The resulting Bell state is entangled, meaning that it cannot be expressed as the tensor product of two individual single-qubit states. Measuring both qubits in the computational basis produces either $|00\rangle$ or $|11\rangle$, each with probability $1/2$.

More generally, for an arbitrary two-qubit pure state

$$|\psi\rangle = a |00\rangle + b |01\rangle + c |10\rangle + d |11\rangle \tag{3.17}$$

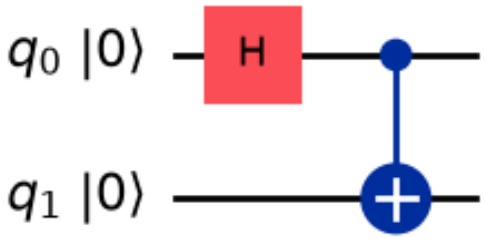


Figure 4: Generation of a Bell state using a Hadamard gate followed by a CNOT gate. The resulting two-qubit state is maximally entangled.

entanglement can be determined by examining the coefficient matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \tag{3.18}$$

The state is separable if and only if A has rank one, or equivalently, its determinant vanishes:

$$\boxed{\begin{cases} ad - bc = 0, & \text{Separable} \\ ad - bc \neq 0, & \text{Entangled} \end{cases}} \tag{3.19}$$

This criterion applies specifically to two-qubit pure states. For the Bell state in Eq. 3.16, $ad - bc = 1/2 \neq 0$, confirming that the state is entangled.

Entangling gates such as CNOT are important building blocks of QNNs, as they enable quantum circuits to establish interactions among qubits that cannot be generated by independent single-qubit operations alone.

3.1.4 Quantum Measurement and Connection to QNNs

Unlike classical computation, where the values of computational variables can generally be accessed directly, extracting information from a quantum system requires measurement.

For a single qubit $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$, a computational-basis measurement yields either 0 or 1, with probabilities

$$P(0) = |\alpha|^2, \qquad P(1) = |\beta|^2 \tag{3.20}$$

A single measurement does not reveal the underlying complex probability amplitudes. Instead, repeated preparation and measurement of the same quantum state allow us to estimate measurement probabilities and expectation values statistically.

In particular, the expectation value of a Hermitian observable $\mathcal{O}$ for a quantum state $|\psi\rangle$ is

$$\langle \mathcal{O} \rangle = \langle \psi | \, \mathcal{O} \, | \psi \rangle \tag{3.21}$$

This provides a natural mechanism for converting quantum information into classical numerical outputs.

The concepts introduced in this section form the foundation of variational QNNs. In a typical QNN, classical input data $\mathbf{x}$ are encoded into quantum states, trainable quantum gates transform these states, entangling operations introduce interactions among qubits, and quantum measurements produce classical outputs.

Mathematically, such a model can be expressed as

$$|\psi(\mathbf{x}, \theta)\rangle = U(\mathbf{x}, \theta) \, |0\rangle^{\otimes n} \tag{3.22}$$

with the corresponding output function

$$f_\theta(\mathbf{x}) = \langle 0|^{\otimes n} \, U^\dagger(\mathbf{x}, \theta) \mathcal{O} U(\mathbf{x}, \theta) \, |0\rangle^{\otimes n} \tag{3.23}$$

where θ denotes the trainable circuit parameters.

Thus, quantum states, unitary operations, entanglement, and measurement collectively provide the building blocks for constructing parameterized quantum models. The following section examines how these principles are combined to form Quantum Neural Networks and how quantum circuits can be used for function approximation.

3.2 Dissection of QNN

Similar to classical neural networks, quantum neural networks (QNN) can also serve as function approximators, with universality requiring suitable encoding and circuit families [10, 12, 13]. Figure 5 shows a **3-qubit** circuit architecture of a standard quantum neural networks. Simply speaking, it is composed of layers of rotation gate (X,Y,Z) and CNOT gate. In the end, the each wire has a measurement gate (usually Z-basis). Let's look at each gate

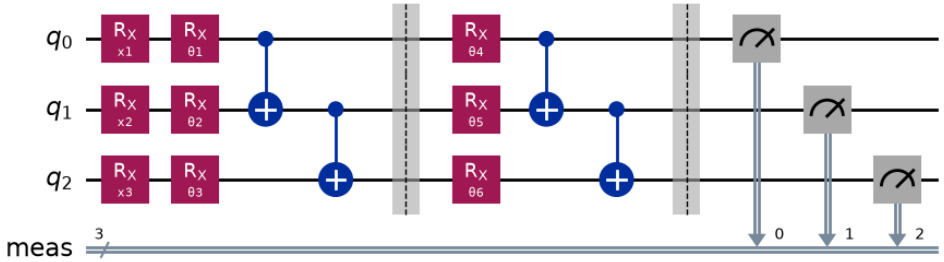


Figure 5: A standard quantum neural network

one by one. Firstly, there are three types of rotation gate R_x, R_y, R_z. Any single-qubit gate is a 2×2 unitary matrix and these three matrices, variants of Pauli-matrices, rotates the vector in the bloch sphere about x, y, z axis by θ degrees. Their mathmatical definition is as follows:

$$R_x(\theta) \equiv e^{-i\theta X/2} = \cos(\frac{\theta}{2})I - i\sin(\frac{\theta}{2})X = \begin{bmatrix} \cos(\frac{\theta}{2}) & -i\sin(\frac{\theta}{2}) \\ -i\sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix}. \tag{3.24}$$

$$R_y(\theta) \equiv e^{-i\theta Y/2} = \cos(\frac{\theta}{2})I - i\sin(\frac{\theta}{2})Y = \begin{bmatrix} \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix}. \tag{3.25}$$

$$R_z(\theta) \equiv e^{-i\theta Z/2} = \cos(\frac{\theta}{2})I - i\sin(\frac{\theta}{2})Z = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}. \tag{3.26}$$

Controlled-NOT (CNOT) gate has two input qubits, known as control qubit and target qubit, respectively. In terms of the computational basis, the action of CNOT is given by $|c\rangle |t\rangle \rightarrow |c\rangle |t \oplus c\rangle$. That is if the control qubit is set to $|1\rangle$ then the target qubit is flipped, otherwise the target qubit is left alone. Thus, in the computational basis $|contro, target\rangle$ the matrix representation of CNOT is:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

For example, if a two-qubit system is in state $|11\rangle$, which in basis vector is $\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$. Here we follow little-endian (LSB)convention ($|q_1 q_0\rangle$) and let $|q_0\rangle$ be the control qubit and $|q_1\rangle$ be the target qubit. When a CNOT gate is applied to it:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = |01\rangle$$

$$CNOT\,|11\rangle \rightarrow |01\rangle$$

These three rotation gates and CNOT gate are also known universal gates in quantum computer, similar to NOR and NAND in classical computer. Up to an overall global phase, any unitary matrix can be decomposed into a sequence of single-qubit rotation gates and CNOT gates [14]. Similar to classical neural network, quantum neural network also has input, hidden and output layers, which corresponds to encoding, rotation and entanglment in Hilbert Space, and measurement. Let's look at them one by one.

3.3 Input Layer: Encoding

For most of machine learning tasks, the data is classical. Therefore, we need a way to encode the classical data into quantum states. Three main encoding approaches are [11]:

1. **Amplitude Encoding**

2. **Basis Encoding**

3. **Angular Encoding**

Of course, there are other more advanced encoding such as Pauli Encoding and IQP Encoding but it's not in the scope of this article. Here, we first talk about the most common encoding method - **Angular Encoding** because it's the easiest to prepare from engineering perspective. Just as the name suggests, the classical data x_i is mapped into a rotation angle using rotation gates. Figure 6 shows how a 3-dimension input vector is embedded into a quantum circuit. Each qubit, represented by a single line here, starts with 0 basis state $|0\rangle$. Then qubit is followed by a rotation gate (can be R_x, R_y, R_z). These rotation gates will then be parametrized by the input values x_1, x_2, x_3. For example, for the first qubit (little-endian), the starting state is $|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. Then the rotation gate, let's say R_x, is parametrized by x_1:

$$R_x(x_1) = \begin{bmatrix} \cos(\frac{x_1}{2}) & -i\sin(\frac{x_1}{2}) \\ -i\sin(\frac{x_1}{2}) & \cos(\frac{x_1}{2}) \end{bmatrix}.$$

Then the qubit is operated as:

$$R_x(x_1)\,|0\rangle = \begin{bmatrix} \cos(\frac{x_1}{2}) & -i\sin(\frac{x_1}{2}) \\ -i\sin(\frac{x_1}{2}) & \cos(\frac{x_1}{2}) \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos(\frac{x_1}{2}) \\ -i\sin(\frac{x_1}{2}) \end{bmatrix}$$

This is equivalent to:

$$|0\rangle \rightarrow \cos(\frac{x_1}{2})\,|0\rangle + (-i\sin(\frac{x_1}{2}))\,|1\rangle$$

If you use R_y, then you will have:

$$R_y(x_1)\,|0\rangle = \begin{bmatrix} \cos(\frac{x_1}{2}) & -\sin(\frac{x_1}{2}) \\ \sin(\frac{x_1}{2}) & \cos(\frac{x_1}{2}) \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos(\frac{x_1}{2}) \\ \sin(\frac{x_1}{2}) \end{bmatrix}$$

Similarly, for the rest of input values x_2, x_3, x_4, the quantum states of the qubits become (assume we use R_x):

$$q_1(x_2) = \begin{bmatrix} \cos(\frac{x_2}{2}) \\ -i\sin(\frac{x_2}{2}) \end{bmatrix}, \quad q_2(x_3) = \begin{bmatrix} \cos(\frac{x_3}{2}) \\ -i\sin(\frac{x_3}{2}) \end{bmatrix}$$

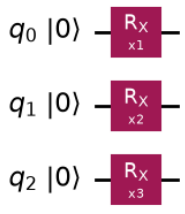


Figure 6: Input layer of a QNN using angluar encoding.

If we look at the 3-qubit system holistically, which is also known as product state, the state transitions from $|000\rangle$ to $|q_2q_1q_0\rangle$. Mathmatically as follows:

$$\begin{bmatrix}1\\0\\0\\0\\0\\0\\0\\0\end{bmatrix} \xrightarrow{\text{angle encoding}} \begin{bmatrix}\cos(\frac{x_3}{2})\\-i\sin(\frac{x_3}{2})\end{bmatrix} \otimes \begin{bmatrix}\cos(\frac{x_2}{2})\\-i\sin(\frac{x_2}{2})\end{bmatrix} \otimes \begin{bmatrix}\cos(\frac{x_1}{2})\\-i\sin(\frac{x_1}{2})\end{bmatrix}$$

This is equivalent to:

$$|q_2q_1q_0\rangle = |q_2\rangle \otimes |q_1\rangle \otimes |q_0\rangle$$

The statevector of entire circuit can be rewritten into tensor product of statevector of each single qubit becuase entanglement has not yet formed.
The tensor product of three 2×1 vectors is a 8×1 vector, which is expanded as follows:

$$\psi(x_3, x_2, x_1) = \begin{bmatrix}\cos(\frac{x_3}{2})\cos(\frac{x_2}{2})\cos(\frac{x_1}{2})\\-i\cos(\frac{x_3}{2})\cos(\frac{x_2}{2})\sin(\frac{x_1}{2})\\-i\cos(\frac{x_3}{2})\sin(\frac{x_2}{2})\cos(\frac{x_1}{2})\\-\cos(\frac{x_3}{2})\sin(\frac{x_2}{2})\sin(\frac{x_1}{2})\\-i\sin(\frac{x_3}{2})\cos(\frac{x_2}{2})\cos(\frac{x_1}{2})\\-\sin(\frac{x_3}{2})\cos(\frac{x_2}{2})\sin(\frac{x_1}{2})\\-\sin(\frac{x_3}{2})\sin(\frac{x_2}{2})\cos(\frac{x_1}{2})\\i\sin(\frac{x_3}{2})\sin(\frac{x_2}{2})\sin(\frac{x_1}{2})\end{bmatrix}$$

If we look each qubit individually, the encoding is a 2×2 matrix. However, if we look the encoding holistically on the entire 3 qubit system, it's a $2^3 \times 2^3 = 8 \times 8$ matrix, which is also a tensor product of all three rotation matrices:

$$\begin{bmatrix}\cos(\frac{x_3}{2}) & -i\sin(\frac{x_3}{2})\\-i\sin(\frac{x_3}{2}) & \cos(\frac{x_3}{2})\end{bmatrix} \otimes \begin{bmatrix}\cos(\frac{x_2}{2}) & -i\sin(\frac{x_2}{2})\\-i\sin(\frac{x_2}{2}) & \cos(\frac{x_2}{2})\end{bmatrix} \otimes \begin{bmatrix}\cos(\frac{x_1}{2}) & -i\sin(\frac{x_1}{2})\\-i\sin(\frac{x_1}{2}) & \cos(\frac{x_1}{2})\end{bmatrix}$$

This makes sense becuase with 3 qubits, the quantum state is in $\mathbb{C}^2 \otimes \mathbb{C}^2 \otimes \mathbb{C}^2 = \mathbb{C}^8$ and the operation shall also be a 8×8 matrix as follows. To make it readable, let's define:

$$c_j = \cos\left(\frac{x_j}{2}\right), \qquad s_j = \sin\left(\frac{x_j}{2}\right), \qquad j = 1, 2, 3, 4.$$

$$U = R_x(x_3) \otimes R_x(x_2) \otimes R_x(x_1)$$

The matrix is as follows:

$$\begin{bmatrix} c_3c_2c_1 & -ic_3c_2s_1 & -ic_3s_2c_1 & -c_3s_2s_1 & -is_3c_2c_1 & -s_3c_2s_1 & -s_3s_2c_1 & is_3s_2s_1 \\ -ic_3c_2s_1 & c_3c_2c_1 & -c_3s_2s_1 & -ic_3s_2c_1 & -s_3c_2s_1 & -is_3c_2c_1 & is_3s_2s_1 & -s_3s_2c_1 \\ -ic_3s_2c_1 & -c_3s_2s_1 & c_3c_2c_1 & -ic_3c_2s_1 & -s_3s_2c_1 & is_3s_2s_1 & -is_3c_2c_1 & -s_3c_2s_1 \\ -c_3s_2s_1 & -ic_3s_2c_1 & -ic_3c_2s_1 & c_3c_2c_1 & is_3s_2s_1 & -s_3s_2c_1 & -s_3c_2s_1 & -is_3c_2c_1 \\ -is_3c_2c_1 & -s_3c_2s_1 & -s_3s_2c_1 & is_3s_2s_1 & c_3c_2c_1 & -ic_3c_2s_1 & -ic_3s_2c_1 & -c_3s_2s_1 \\ -s_3c_2s_1 & -is_3c_2c_1 & is_3s_2s_1 & -s_3s_2c_1 & -ic_3c_2s_1 & c_3c_2c_1 & -c_3s_2s_1 & -ic_3s_2c_1 \\ -s_3s_2c_1 & is_3s_2s_1 & -is_3c_2c_1 & -s_3c_2s_1 & -ic_3s_2c_1 & -c_3s_2s_1 & c_3c_2c_1 & -ic_3c_2s_1 \\ is_3s_2s_1 & -s_3s_2c_1 & -s_3c_2s_1 & -is_3c_2c_1 & -c_3s_2s_1 & -ic_3s_2c_1 & -ic_3c_2s_1 & c_3c_2c_1 \end{bmatrix} \tag{3.27}$$

Please be aware that since we use little-endian convention, the tensor product of operation matrix follows the same order as the qubit. If our initial state is $|0\rangle$, then the resulting state is the first column of this matrix. In fact, the matrix is

$$U_{\mathbf{b},\mathbf{b}'} = (-i)^{d_H(\mathbf{b},\mathbf{b}')} \prod_{j:b_j=b'_j} c_j \prod_{j:b_j\neq b'_j} s_j$$

where $d_H(\mathbf{b}, \mathbf{b}')$ is Hamming distance between target state and initial state.
Before concluding angle encoding, there is one more thing - **scaling**. As you might have observed, the trignometric functions are all periodic (2π) meaning that

$$|\psi(x)\rangle = |\psi(x + 2\pi)\rangle$$

If the range of classical data is larger than 2π, then two different classical values will be mapped to the same quantum state, causing information loss. Therefore, scaling must be conducted such that the input values, before encoding, must be between 0 and π. This is becuase the periodicity of cosine and sine functions are 2π, so the periodicity in $\cos(\frac{x}{2})$ is π. A simple and most common way to do scaling is as follows:

$$\theta = \pi \frac{x - x_{min}}{x_{max} - x_{min}}$$

Now, the features are mapped into a non-repeatitive and continuous angle space. Finally, there are many variants of angle encoding such as XYZ encoding, data-reuploading, multiscale, and adaptive encoding. What has to be aware of is that encoding can have notrivial impact on the representation capacity of the model [13, 12].

3.4 Hidden Layer - Rotation

Now, after angle encoding, each qubit is in the state

$$|\psi\rangle \cos(\frac{x_i}{2}) |0\rangle + (-i \sin(\frac{x_i}{2})) |1\rangle = \begin{bmatrix} \cos(\frac{x_1}{2}) \\ -i \sin(\frac{x_1}{2}) \end{bmatrix}.$$

Then, this qubit will go through another rotation gate, let's say R_x, which is parametrized by θ, resembling the weight in classical neural networks. The state becomes:

$$R_x(\theta)\,|\psi\rangle = \begin{bmatrix} \cos(\frac{\theta}{2}) & -i\sin(\frac{\theta}{2}) \\ -i\sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix} \times \begin{bmatrix} \cos(\frac{x_i}{2}) \\ -i\sin(\frac{x_i}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta}{2})\cos(\frac{x_i}{2}) - \sin(\frac{\theta}{2})\sin(\frac{x_i}{2}) \\ -i\sin(\frac{\theta}{2})\cos(\frac{x_i}{2}) - i\cos(\frac{\theta}{2})\sin(\frac{x_i}{2}) \end{bmatrix}$$

Using following trignometric identity, we can further simplify.

$$\cos(A+B) = \cos(A)\cos(B) - \sin(A)\sin(B)$$
$$\sin(A+B) = \sin(A)\cos(B) + \cos(A)\sin(B)$$

$$\begin{bmatrix} \cos(\frac{\theta}{2})\cos(\frac{x_i}{2}) - \sin(\frac{\theta}{2})\sin(\frac{x_i}{2}) \\ -i\sin(\frac{\theta}{2})\cos(\frac{x_i}{2}) - i\cos(\frac{\theta}{2})\sin(\frac{x_i}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta+x_i}{2}) \\ -i\sin(\frac{\theta+x_i}{2}) \end{bmatrix}$$

For example, in Figure 7, the first qubit becomes:

$$|q_0\rangle = \begin{bmatrix} \cos(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix}.$$

For the rest of qubits, the states becomes:

$$q_1 = \begin{bmatrix} \cos(\frac{\theta_2+x_2}{2}) \\ -i\sin(\frac{\theta_2+x_2}{2}) \end{bmatrix}, \quad q_2 = \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2}) \end{bmatrix}$$

The entire 3-qubit system then becomes the tensor product of these four vectors:

$$\psi_{q_3q_2q_1} = \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_2+x_2}{2}) \\ -i\sin(\frac{\theta_2+x_2}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix}$$

Which can be expaned as:

$$\begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} \tag{3.28}$$

Similarily, the operation matrix on these 3-qubits system is also the tensor product of these three 2×2 matrices:

$$U_{hidden} = \begin{bmatrix} \cos(\frac{\theta_3}{2}) & -i\sin(\frac{\theta_3}{2}) \\ -i\sin(\frac{\theta_3}{2}) & \cos(\frac{\theta_3}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_2}{2}) & -i\sin(\frac{\theta_2}{2}) \\ -i\sin(\frac{\theta_2}{2}) & \cos(\frac{\theta_2}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_1}{2}) & -i\sin(\frac{\theta_1}{2}) \\ -i\sin(\frac{\theta_1}{2}) & \cos(\frac{\theta_1}{2}) \end{bmatrix}$$

Similarily, if we define:

$$c_j = \cos(\frac{\theta_j}{2}), \quad s_j = \sin(\frac{\theta_j}{2}), \quad j = 1,2,3,4$$

Then, this 8×8 matrix is the same as Eq. (3.27)

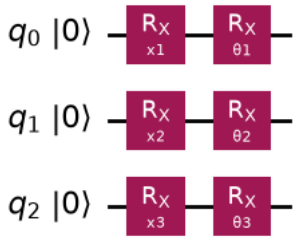


Figure 7: First hidden layer of rotation gates

3.5 Hidden Layer - Entanglement

Local rotations and entangling gates are standard building blocks of parameterized quantum circuits [11]. What's getting more interesting is the CNOT gate in the next layer. If we only apply rotation gates on each qubit separately, there is no interactions between qubits. CNOT gate allows interactions between qubits so that information can be passed across qubits instead of just staying at only a single qubit. A good analogy is that in classical neural networks, each neuron takes all the neurons in the previous layer as inputs, if we assume fully-connected layers.

Although we have a 3-qubit system, but let's look at the first two qubits q_0, q_1. In Figure 8, after encoding layer and first rotation layer, the state of each qubit is:

$$\begin{aligned} |q_0\rangle &= \cos(\frac{\theta_1 + x_1}{2}) |0\rangle + (-i \sin(\frac{\theta_1 + x_1}{2})) |1\rangle \\ |q_1\rangle &= \cos(\frac{\theta_2 + x_2}{2}) |0\rangle + (-i \sin(\frac{\theta_2 + x_2}{2})) |1\rangle \end{aligned} \tag{3.29}$$

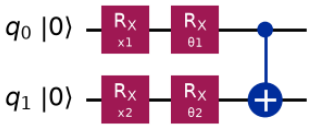


Figure 8: CNOT gate layer on two qubits after rotation gates

The state of entire system is:

$$|q_1 q_0\rangle = \begin{bmatrix} \cos(\frac{\theta_1+x_1}{2}) \cos(\frac{\theta_2+x_2}{2}) \\ -i \cos(\frac{\theta_2+x_2}{2}) \sin(\frac{\theta_1+x_1}{2}) \\ -i \cos(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \\ -\sin(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \end{bmatrix} \tag{3.30}$$

If we apply a CNOT gate on these two qubits, then we have:

$$CNOT\,|q_1q_0\rangle = \begin{bmatrix}1&0&0&0\\0&0&0&1\\0&0&1&0\\0&1&0&0\end{bmatrix} \times \begin{bmatrix}\cos(\frac{\theta_1+x_1}{2})\cos(\frac{\theta_2+x_2}{2})\\-i\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2})\\-i\cos(\frac{\theta_1+x_1}{2})\sin(\frac{\theta_2+x_2}{2})\\-\sin(\frac{\theta_1+x_1}{2})\sin(\frac{\theta_2+x_2}{2})\end{bmatrix} = \begin{bmatrix}\cos(\frac{\theta_1+x_1}{2})\cos(\frac{\theta_2+x_2}{2})\\-\sin(\frac{\theta_1+x_1}{2})\sin(\frac{\theta_2+x_2}{2})\\-i\cos(\frac{\theta_1+x_1}{2})\sin(\frac{\theta_2+x_2}{2})\\-i\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2})\end{bmatrix} \tag{3.31}$$

As you might observe, the second element and the fourth element of state vector got swapped. Now, these two qubits are entangled! This means that the system cannot be rewriten into a tensor product of two single qubit anymore. You can easily verify this by yourself using Eq. **??**.

$$CNOT\,|q_1q_0\rangle \neq |q_1\rangle \otimes |q_0\rangle$$

Now, if take the third qubit into consideration and look at the entire system, the operation matrix is:

$$U_{entangle} = I \otimes CNOT_{q_0 \to q_1} \tag{3.32}$$

where I is 2×2 identity matrix. To expand, we have

$$\begin{aligned} U_{entangle} &= \begin{bmatrix}1&0\\0&1\end{bmatrix} \otimes \begin{bmatrix}1&0&0&0\\0&0&0&1\\0&0&1&0\\0&1&0&0\end{bmatrix} \\ &= \begin{bmatrix}1&0&0&0&0&0&0&0\\0&0&0&1&0&0&0&0\\0&0&1&0&0&0&0&0\\0&1&0&0&0&0&0&0\\0&0&0&0&1&0&0&0\\0&0&0&0&0&0&0&1\\0&0&0&0&0&0&1&0\\0&0&0&0&0&1&0&0\end{bmatrix} \end{aligned} \tag{3.33}$$

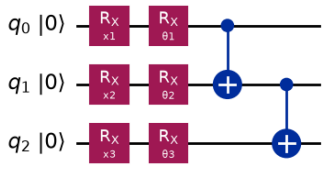


Figure 9: Entanglement layer in QNN

Then, we can apply this matrix 3.33 on the statevector we derived from Eq 3.28:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} \tag{3.34}$$

Please notice that the original order $(1, 2, 3, 4, \quad 5, 6, 7, 8)$ becomes $(1, 4, 3, 2, \quad 5, 8, 7, 6)$. For every "subgroup", the second and fourth element are swapped.
Then, as Figure 9 shows, we apply CNOT gate on the second and third qubits q_1, q_2, the operation matrix is:

$$U_{entangle} = CNOT_{q_1 \to q_2} \otimes I$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Then, we apply this matrix on the statevector:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} \tag{3.35}$$

If you look carefully, What happens is that compare with Eq. 3.34, swapping happens as follows:

$$3 \leftrightarrow 7, \quad 4 \leftrightarrow 8$$

As you might observe, you can apply a CNOT gate on any of the two qubits (e.g., q_0, q_2) and CNOT gates are essentially just permutation matrix that induces a permutation of amplitude in statevector. The magic is that all these four qubits are entangled meaning the statevector is not simply a tensor product of four single-qubit statevector:

$$|q_2 q_1 q_0\rangle \neq |q_2\rangle \otimes |q_1\rangle \otimes |q_0\rangle$$

3.6 Information Propogation through CNOT Gates

The importance of CNOT gate is allowing information be propogatied from control qubit to target qubit, if we measure in Z-basis. Let's look at a 2-qubit system first. Previously, we have derived states of 2-qubit system in Eq. 3.30 that after a layer of encoding and rotation. The probability of observing $q_1 = 0$ is as follows.

$$\begin{aligned} \text{Let} \quad \alpha &= \frac{x_1 + \theta_1}{2}, \quad \beta = \frac{x_2 + \theta_2}{2} \\ P(|q_1\rangle = |0\rangle) &= \cos^2(\alpha)\cos^2(\beta) + \cos^2(\beta)\sin^2(\alpha) \\ &= \cos^2(\beta)(\cos^2(\alpha) + \sin^2(\alpha)) \\ &= \cos^2(\beta) = \cos(\frac{x_2 + \theta_2}{2}) \end{aligned} \tag{3.36}$$

As you can see, the probability of each state in q_1 is a function of $x2$ and θ_2:

$$P(|q_1\rangle = |0\rangle) = f(x_2, \theta_2)$$

However, after a CNOT gate, the statevector becomes Eq. 3.31. The The probability of observing $q_1 = 0$ becomes as follwoing.

$$\begin{aligned} P(|q_1\rangle = |0\rangle) &= \cos^2(\alpha)\cos^2(\alpha) + \sin^2(\alpha)\sin^2(\beta) \\ &= \cos^2(\frac{x_1 + \theta_1}{2})\cos^2(\frac{x_2 + \theta_2}{2}) + \sin^2(\frac{x_1 + \theta_1}{2})\sin^2(\frac{x_2 + \theta_2}{2}) \\ &= f(x_1, x_2, \theta_1, \theta_2) \end{aligned} \tag{3.37}$$

Now, as you can see, the probability amplitude of q_1 becomes a function of $x_1, x_2, \theta_1, \theta_2$! The information from q_0, the first qubit, both input value x_1 and parameters θ_1 are passed down to q_1. This gives the circuits much greater expressivity. However, another thing to notice is that the probability of q_0 does not change before or after CNOT gate.

$$P(|q_0\rangle = |0\rangle) = \cos^2(\frac{\theta_1 + x_1}{2}) \tag{3.38}$$

Intrestingly, the infomration did not pass from target to control. However, this is given we measure in Z-basis. Measurement in other basis will lead to different sitautions.
Again, if we compare with classical neural networks, it's a function of all the input values as Eq. 3.39 shows. This is also knonw as fully connected layers. If the output of a classical Nueral Networks is function of only a single input x_i, it will not be able learn the latent

structure of the data, significantly harming the expressivity. The entanglement layer tries to do the similar thing.

$$\begin{aligned} f_{NN} &= f(\sigma(W^T X + b)) \\ &= f(\sigma(\sum_{n=1}^{N} w_n x_n + b)) \\ &= f(x_1, x_2, \cdots, x_N, W) \end{aligned} \tag{3.39}$$

Now, let's look at the 3-qubit system. Before the entanglment layer, the probability of observing the fourth qubit $|q_2\rangle = |0\rangle$ can be derived from Eq. 3.28 as follows:

$$\begin{aligned} \text{Let} \alpha_j &= \frac{x_j + \theta_j}{2}, \quad j = 1, 2, 3 \\ P(q_2 = 0) &= |\psi_{000}|^2 + |\psi_{001}|^2 + |\psi_{010}|^2 + |\psi_{011}|^2 \\ &= \cos^2(\alpha_3)\cos^2(\alpha_2)\cos^2(\alpha_1) + \cos^2(\alpha_3)\cos^2(\alpha_2)\sin^2(\alpha_1) \\ &+ \cos^2(\alpha_3)\sin^2(\alpha_2)\cos^2(\alpha_1) + \cos^2(\alpha_3)\sin^2(\alpha_2)\sin^2(\alpha_1) \\ &= \cos^2(\alpha_3)[\cos^2(\alpha_2)(\cos^2(\alpha_1) + \sin^2(\alpha_1)) + \sin^2(\alpha_2)(\cos^2(\alpha_1) + \sin^2(\alpha_1))] \\ &= \cos^2(\alpha_3) \\ &= \cos^2(\frac{\theta_3 + x_3}{2}) \end{aligned} \tag{3.40}$$

As you can see, without CNOT (entanglement) layer, regardless how many rotation gate you add to the third qubit q_2 (this also applied to other qubits), the probability amplitude for each qubit is just a function of its encoding input x_3 and the parameters of the rotation gate in its own path θ_3. Mathmatically, it's $P(q_2 = 0) = f(\theta_3, x_3)$ The information will never be propagated across the lines. However, after the first entanglement layer, according to Eq. 3.35, the the probability amplitude of $|q_3\rangle = |0\rangle$ becomes as follows:

$$\begin{aligned} P(q_2 = 0) &= \cos^2(\alpha_3)\cos^2(\alpha_2)\cos^2(\alpha_1) + \cos^2(\alpha_3)\sin^2(\alpha_2)\sin^2(\alpha_1) \\ &+ \sin^2(\alpha_3)\sin^2(\alpha_2)\cos^2(\alpha_1) + \sin^2(\alpha_3)\cos^2(\alpha_2)\sin^2(\alpha_1) \\ &= \cos^2(\alpha_3)[\cos^2(\alpha_2)\cos^2(\alpha_1) + \sin^2(\alpha_2)\sin^2(\alpha_1)] \\ &+ \sin^2(\alpha_3)[\sin^2(\alpha_2)\cos^2(\alpha_1) + \cos^2(\alpha_2)\sin^2(\alpha_1)] \\ &= \frac{\cos^2(\alpha_3)}{2}[1 + \cos(2\alpha_2)\cos(2\alpha_1)] + \frac{\cos^2(\alpha_3)}{2}[1 - \cos(2\alpha_2)\cos(2\alpha_1)] \\ &= \frac{1}{2}[1 + \cos(2\alpha_3)\cos(2\alpha_2)\cos(2\alpha_1)] \\ &= \frac{1}{2}[1 + \cos(\theta_3 + x_3)\cos(\theta_2 + x_2)\cos(\theta_1 + x_1)] \\ &= \frac{1}{2}[1 + \prod_{j=1}^{3}\cos(\theta_j + x_j)] \end{aligned} \tag{3.41}$$

If we look back at, it's clear that

$$\begin{aligned} P_{\text{before}}(q_2 = 0) &= \cos^2(\frac{\theta_3 + x_3}{2}) \\ P_{\text{after}}(q_2 = 0) &= \frac{1}{2}[1 + \prod_{j=1}^{3}\cos(\theta_j + x_j)] \end{aligned} \tag{3.42}$$

Now, you can clearly see what magic the entanglment layer has done. If you measure q_3 before entanglment, its probability amplitude is determined by only θ_4, x_4 whereas after entanglment it's determined by all the inputs and parameters. As said before, after entanglment, the statevector cannot be rewritten into tensor product of statevector of each single qubit. Therefore, all the computation after this point will be holistic, meaning that we need to do $(2^n \times 2^n) \times (2^n \times 1)$ matrix-vector calculation at each layer, which is a big challenge for classical computer. Imagine if you have 30 qubits, the dimension is 1,073,741,924. None of the CPUs, GPUs, or TPUs has enough RAM to do this calculation. However, it can be easily done by a qunatum computer, in fact IBM machines such as Nighthawk or Heron family offer 120 qubits and this number will be increasing in the next couple of years. In terms of large matrix calculation, the quantum advantage is clear.
Alright, let's keep watching how this quantum neural network evolve. As Figure 10 shows, another rotation layer is applied after entanglement layer, as the second hidden layer. The rotation gates are parametrized as $\theta_4, \theta_5, \theta_6$ respectively. The operation matrix of this layer

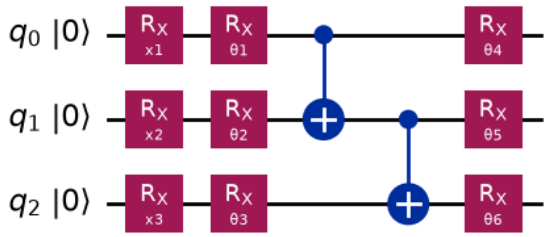


Figure 10: Quantum Neural Network with 2 hidden layers

is then:

$$
\begin{aligned}
U_{hidden} &= R_x(\theta_6) \otimes R_x(\theta_5) \otimes R_x(\theta_4) \\
&= \begin{bmatrix} \cos(\frac{\theta_6}{2}) & -i\sin(\frac{\theta_6}{2}) \\ -i\sin(\frac{\theta_6}{2}) & \cos(\frac{\theta_6}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_5}{2}) & -i\sin(\frac{\theta_5}{2}) \\ -i\sin(\frac{\theta_5}{2}) & \cos(\frac{\theta_5}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_4}{2}) & -i\sin(\frac{\theta_4}{2}) \\ -i\sin(\frac{\theta_4}{2}) & \cos(\frac{\theta_4}{2}) \end{bmatrix} \\
&= \begin{bmatrix}
c_6c_5c_4 & -ic_6c_5s_4 & -ic_6s_5c_4 & -c_6s_5s_4 & -is_6c_5c_4 & -s_6c_5s_4 & -s_6s_5c_4 & is_6s_5s_4 \\
-ic_6c_5s_4 & c_6c_5c_4 & -c_6s_5s_4 & -ic_6s_5c_4 & -s_6c_5s_4 & -is_6c_5c_4 & is_6s_5s_4 & -s_6s_5c_4 \\
-ic_6s_5c_4 & -c_6s_5s_4 & c_6c_5c_4 & -ic_6c_5s_4 & -s_6s_5c_4 & is_6s_5s_4 & -is_6c_5c_4 & -s_6c_5s_4 \\
-c_6s_5s_4 & -ic_6s_5c_4 & -ic_6c_5s_4 & c_6c_5c_4 & is_6s_5s_4 & -s_6s_5c_4 & -s_6c_5s_4 & -is_6c_5c_4 \\
-is_6c_5c_4 & -s_6c_5s_4 & -s_6s_5c_4 & is_6s_5s_4 & c_6c_5c_4 & -ic_6c_5s_4 & -ic_6s_5c_4 & -c_6s_5s_4 \\
-s_6c_5s_4 & -is_6c_5c_4 & is_6s_5s_4 & -s_6s_5c_4 & -ic_6c_5s_4 & c_6c_5c_4 & -c_6s_5s_4 & -ic_6s_5c_4 \\
-s_6s_5c_4 & is_6s_5s_4 & -is_6c_5c_4 & -s_6c_5s_4 & -ic_6s_5c_4 & -c_6s_5s_4 & c_6c_5c_4 & -ic_6c_5s_4 \\
is_6s_5s_4 & -s_6s_5c_4 & -s_6c_5s_4 & -is_6c_5c_4 & -c_6s_5s_4 & -ic_6s_5c_4 & -ic_6c_5s_4 & c_6c_5c_4
\end{bmatrix}
\end{aligned}
\tag{3.43}
$$

The new statevector becomes:

$$U_{hidden} \times \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} \tag{3.44}$$

Firstly, let's define:

$$\begin{aligned} C_1 &= \cos(\frac{\theta_1+x_1}{2}), \quad S_1 = \sin(\frac{\theta_1+x_1}{2}) \\ C_2 &= \cos(\frac{\theta_2+x_2}{2}), \quad S_2 = \sin(\frac{\theta_2+x_2}{2}) \\ C_3 &= \cos(\frac{\theta_3+\theta_6+x_3}{2}), \quad S_3 = \sin(\frac{\theta_3+\theta_6+x_3}{2}) \\ c_j &= \cos(\frac{\theta_j}{2}), \quad s_j = \sin(\frac{\theta_j}{2}) \end{aligned}$$

$$|q_2q_1q_0\rangle = \begin{bmatrix} C_3c_5(c_4C_2C_1 + is_4S_2S_1) + S_3s_5(ic_4S_2C_1 + s_4C_2S_1) \\ C_3c_5(-is_4C_2C_1 - c_4S_2S_1) + S_3s_5(s_4S_2C_1 + ic_4C_2S_1) \\ C_3s_5(-ic_4C_2C_1 + s_4S_2S_1) + S_3c_5(-c_4S_2C_1 + is_4C_2S_1) \\ C_3s_5(-s_4C_2C_1 + ic_4S_2S_1) + S_3c_5(is_4S_2C_1 - c_4C_2S_1) \\ S_3c_5(-ic_4C_2C_1 + s_4S_2S_1) + C_3s_5(-c_4S_2C_1 + is_4C_2S_1) \\ S_3c_5(-s_4C_2C_1 + ic_4S_2S_1) + C_3s_5(is_4S_2C_1 - c_4C_2S_1) \\ S_3s_5(-c_4C_2C_1 - is_4S_2S_1) + C_3c_5(-ic_4S_2C_1 - s_4C_2S_1) \\ S_3s_5(is_4C_2C_1 + c_4S_2S_1) + C_3c_5(-s_4S_2C_1 - ic_4C_2S_1) \end{bmatrix} \tag{3.45}$$

However, as the circuit depth increases, manually deriving the statevector quickly becomes computationally intractable. This naturally raises an important question: can we abstract these increasingly complex expressions into a more compact mathematical representation that allows us to analyze the QNN systematically?

Fortunately, the answer is yes! In fact, under the data-encoding schemes considered here, the output of a QNN can be represented as a **finite Fourier series** [13].

To understand why, let us revisit Eq. 3.45. A closer examination reveals that each probability amplitude consists of combinations of sine and cosine functions. By applying standard trigonometric identities, these expressions can be rewritten in the general form

$$a_j(x,\theta) = \sum_{\alpha\in\Lambda} [A_{j,\alpha}(\theta)\cos(\alpha x) + B_{j,\alpha}(\theta)\sin(\alpha x)] \tag{3.46}$$

where Λ denotes a finite set of frequencies, and $A_{j,\alpha}(\theta)$ and $B_{j,\alpha}(\theta)$ are coefficients determined by the trainable parameters.

This is precisely the structure of a finite trigonometric series! More importantly, since the QNN output is obtained through the expectation value of an observable, it can also be expressed as a finite Fourier series:

$$f_\theta(x) = \sum_{\omega \in \Omega} c_\omega(\theta) e^{i\omega x} \tag{3.47}$$

Here, Ω denotes the set of accessible frequencies. Unlike classical neural networks with learnable frequencies, such as SIREN [15], the accessible frequency spectrum of a QNN is primarily determined by its data-encoding strategy and circuit architecture. In particular, both circuit depth, characterized by the number of data-reuploading layers, and circuit width, characterized by the number of qubits involved in data encoding, can influence the accessible frequency spectrum [13]. We will explore these relationships in greater detail in the following sections.

Furthermore, $\omega \in \Omega$ represents an individual frequency component, while $c_\omega(\theta)$ denotes its corresponding trainable Fourier coefficient. From Fourier analysis, we know that the expressive power of a Fourier series—namely, the range of functions it can approximate—depends in part on the richness of its accessible frequency spectrum. Unlike in SIREN, where frequencies can be learned during training, the frequency spectrum of a QNN with a fixed encoding scheme is predetermined by its architecture rather than directly optimized through training.

This naturally raises an intriguing question: **Is a predetermined frequency spectrum a fundamental limitation of QNNs, or could it actually become an advantage?** We will investigate this question in the following sections.

This remarkable connection allows us to study the expressive power of QNNs through their accessible frequency spectra and trainable Fourier coefficients, rather than explicitly tracking the exponentially growing quantum statevector.

3.7 Output Layer: Quantum Measurement

The final stage of a QNN is the measurement layer, which converts quantum information into classical outputs. Unlike classical neural networks, where intermediate activations can be directly accessed, the quantum statesevolving throughout a quantum circuit cannot be directly observed without performing measurements. In particular, a computational-basis measurement of a qubit yields either 0 or 1, with probabilities governed by the Born rule [4, Sec. 2.2]. However, the complex probability amplitudes underlying these outcomes cannot be directly recovered from a single measurement.

How, then, can we extract meaningful numerical outputs from a quantum circuit? A common approach is to measure the expectation value of a quantum observable $\mathcal{O}$.

After the encoding layer and multiple variational rotation and entangling layers, the quantum state can be expressed as

$$|\psi(x,\theta)\rangle = U(x,\theta)\,|0\rangle^{\otimes n} \tag{3.48}$$

where $U(x, \theta)$ represents the overall unitary transformation of the circuit. Despite the potentially complex sequence of quantum operations, the entire ideal circuit can be mathematically represented by a single unitary operator.

Given an observable $\mathcal{O}$, the output of the QNN is defined as its expectation value:

$$\begin{aligned} f_\theta(x) &= \langle \psi(x,\theta) | \, \mathcal{O} \, | \psi(x,\theta) \rangle \\ &= \langle 0 |^{\otimes n} \, U^\dagger(x,\theta) \mathcal{O} U(x,\theta) \, | 0 \rangle^{\otimes n} \end{aligned} \tag{3.49}$$

One of the most commonly used observables in variational QNNs is the Pauli-Z operator:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \tag{3.50}$$

To understand how this measurement works, consider a single qubit in the state

$$|\psi\rangle = \alpha \, |0\rangle + \beta \, |1\rangle \, , \qquad |\alpha|^2 + |\beta|^2 = 1 \tag{3.51}$$

According to the Born rule, the probabilities of measuring the qubit in states $|0\rangle$ and $|1\rangle$ are

$$P(0) = |\alpha|^2, \qquad P(1) = |\beta|^2$$

Since the eigenvalues of the Pauli-Z operator are $+1$ and -1, its expectation value is

$$\begin{aligned} \langle Z \rangle &= \langle \psi | \, Z \, | \psi \rangle \\ &= |\alpha|^2 - |\beta|^2 \\ &= P(0) - P(1) \\ &= 2P(0) - 1 \end{aligned} \tag{3.52}$$

Consequently, the measurement probabilities can be recovered from the expectation value:

$$P(0) = \frac{1 + \langle Z \rangle}{2}, \qquad P(1) = \frac{1 - \langle Z \rangle}{2} \tag{3.53}$$

Since $-1 \leq \langle Z \rangle \leq 1$, an expectation value closer to $+1$ indicates a higher probability of measuring $|0\rangle$, whereas a value closer to -1 indicates a higher probability of measuring $|1\rangle$.

Importantly, the expectation value should not be confused with the probability amplitude. While the probability amplitudes α and β are generally complex-valued, the expectation value $\langle Z \rangle$ is real-valued and reflects the difference between the two measurement probabilities.

On real quantum hardware, the expectation value cannot be obtained from a single circuit execution. Instead, the circuit must be executed repeatedly to estimate the expectation value statistically [10, 11]:

$$\widehat{\langle Z \rangle} = \frac{N_0 - N_1}{N} \tag{3.54}$$

where N_0 and N_1 denote the numbers of measurement outcomes 0 and 1, respectively, and $N = N_0 + N_1$ is the total number of measurement shots.

For a multi-qubit system, the same principle applies. Consider the three-qubit QNN introduced earlier. Following the little-endian convention, its final statevector is

$$|\psi\rangle = \begin{bmatrix} a_{000} \\ a_{001} \\ a_{010} \\ a_{011} \\ a_{100} \\ a_{101} \\ a_{110} \\ a_{111} \end{bmatrix} \tag{3.55}$$

where

$$\sum_{i,j,k\in\{0,1\}} |a_{ijk}|^2 = 1 \tag{3.56}$$

To measure the first qubit, q_1, we define the observable

$$\mathcal{O}_1 = I \otimes I \otimes Z \tag{3.57}$$

The corresponding expectation value becomes

$$\begin{aligned} \langle Z_1 \rangle &= \langle \psi | \, \mathcal{O}_1 \, | \psi \rangle \\ &= |a_{000}|^2 - |a_{001}|^2 \\ &\quad + |a_{010}|^2 - |a_{011}|^2 \\ &\quad + |a_{100}|^2 - |a_{101}|^2 \\ &\quad + |a_{110}|^2 - |a_{111}|^2 \end{aligned} \tag{3.58}$$

This expression illustrates that measuring a single qubit generally involves contributions from multiple computational basis states. In particular, the expectation value depends on the marginal measurement probabilities associated with the selected qubit, rather than on any individual probability amplitude.

Similarly, the remaining two qubits can be measured using the observables

$$\mathcal{O}_2 = I \otimes Z \otimes I, \qquad \mathcal{O}_3 = Z \otimes I \otimes I \tag{3.59}$$

By measuring all three qubits, the QNN produces a vector of expectation values:

$$\mathbf{f}_\theta(x) = \begin{bmatrix} \langle Z_1 \rangle \\ \langle Z_2 \rangle \\ \langle Z_3 \rangle \end{bmatrix} \tag{3.60}$$

These expectation values serve as classical output features and can either be used directly for prediction or passed to an additional classical processing layer.

Beyond Pauli-Z measurements, other observables, such as Pauli-X, Pauli-Y, and multi-qubit correlation operators, can also be employed to extract different properties of the quantum state. However, a detailed discussion of these measurement strategies is beyond the scope of this section.

In summary, quantum measurement provides the essential bridge between the quantum representation and the classical output of a QNN. More importantly, expressing the QNN

output as the expectation value of an observable establishes the mathematical foundation for analyzing its function representation, particularly through Fourier analysis.

4 Fourier Analysis of Quanutm Neural Networks and Expressivity

4.1 Basics of Fourier Series and Fourier Transform

Fourier analysis provides a powerful mathematical framework for representing and analyzing functions in terms of their frequency components. Standard treatments cover both Fourier series and Fourier transforms [16]. The fundamental idea is that a function can be decomposed into a combination of oscillatory basis functions, such as sines, cosines, or complex exponentials. Instead of examining a function solely in its original domain, Fourier analysis allows us to study its behavior in the frequency domain.

Two fundamental tools in Fourier analysis are the *Fourier series* and the *Fourier transform*. Although closely related, they serve different purposes and apply to different classes of functions.

Fourier Series

A Fourier series represents a periodic function as an infinite sum of sinusoidal components with different frequencies, amplitudes, and phases.

For a periodic function $f(x)$ with period T, its Fourier series can be expressed in complex exponential form as

$$f(x) = \sum_{k=-\infty}^{\infty} c_k e^{ik\omega_0 x} \tag{4.1}$$

where $\omega_0 = 2\pi/T$ is the fundamental angular frequency, $k \in \mathbb{Z}$ is the harmonic index, and c_k denotes the Fourier coefficient associated with frequency $k\omega_0$.

Each Fourier coefficient can be calculated by projecting the original function onto its corresponding complex exponential basis:

$$c_k = \frac{1}{T} \int_0^T f(x) e^{-ik\omega_0 x}\, dx \tag{4.2}$$

These coefficients describe the contribution of each frequency component to the original function. In particular, $|c_k|$ determines its magnitude, while the complex phase of c_k determines its phase shift.

According to Euler's formula,

$$e^{i\omega x} = \cos(\omega x) + i \sin(\omega x) \tag{4.3}$$

the complex exponential representation is equivalent to expressing a real-valued periodic function as a sum of sine and cosine functions.

Importantly, different frequencies capture different levels of variation in a function. Low-frequency components describe slowly varying patterns, whereas high-frequency components represent rapid oscillations and fine-grained details.

In practical applications, Fourier series are often approximated using a finite number of frequency components. More generally, a finite trigonometric expansion can be written as

$$f(x) = \sum_{\omega \in \Omega} c_\omega e^{i\omega x} \tag{4.4}$$

where Ω is a finite set of frequencies. Unlike the conventional Fourier series, thefrequencies in this generalized representation are not necessarily integer multiples of a common fundamental frequency.

Fourier Transform

While Fourier series decompose periodic functions into discrete harmonics, the Fourier transform extends frequency-domain analysis to nonperiodic functions.

For a sufficiently well-behaved function $f(x)$, the continuous Fourier transform is defined as

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(x) e^{-i\omega x}\, dx \tag{4.5}$$

where $\omega \in \mathbb{R}$ represents the angular frequency and $\hat{f}(\omega)$ describes the frequency-domain representation of $f(x)$.

The original function can be reconstructed using the inverse Fourier transform:

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(\omega) e^{i\omega x}\, d\omega \tag{4.6}$$

The key distinction is that the Fourier series uses a discrete set of frequencies, whereas the Fourier transform is defined over a continuous frequency variable. Nevertheless, both share the same fundamental principle: a function can be understood through its frequency components and their corresponding coefficients.

From a function-approximation perspective, the frequency spectrum reveals which oscillatory patterns are present in a function. This perspective is particularly useful when studying the expressive power of machine learning models.

4.2 Connection to Quantum Neural Networks

The significance of Fourier analysis becomes apparent when examining the mathematical representation of Quantum Neural Networks (QNNs).

As discussed previously, the output of a variational QNN is typically defined through the expectation value of an observable:

$$f_\theta(x) = \langle 0 | U^\dagger(x, \theta) \mathcal{O} U(x, \theta) | 0 \rangle \tag{4.7}$$

For the finite-dimensional quantum circuits with data-encoding operations considered in this paper, this expectation-value function can be expressed as a finite Fourier series [13]:

$$f_\theta(x) = \sum_{\omega \in \Omega} c_\omega(\theta) e^{i\omega x} \tag{4.8}$$

Here, Ω denotes the set of accessible frequencies, while $c_\omega(\theta)$ represents the Fourier coefficients controlled by the trainable circuit parameters.

Unlike conventional Fourier approximations, where the selected frequencies may be chosen or optimized for a particular task, the accessible frequencies of a QNN with a fixed data-encoding scheme are determined by its encoding operations and circuit architecture. The trainable parameters primarily control the corresponding Fourier coefficients, subject to constraints imposed by the circuit [13].

Therefore, Fourier analysis provides a natural framework for investigating two fundamental aspects of QNN expressivity: which frequencies a quantum circuit can represent, and how flexibly it can adjust their coefficients.

In the following section, we will examine how these properties emerge from quantum data encoding and how circuit depth and width influence the accessible frequency spectrum.

4.3 Quantum Neural Networks as Finite Fourier Series

This section presents a detailed derivation of a result due to Schuld, Sweke, and Meyer [13]: the output of a quantum neural network (QNN) in which the input is encoded through Hamiltonian-generated gates can be expressed as a finite Fourier series in that input. The argument proceeds in four stages. First, we fix the general form of the circuit and show that the encoding gates may, without loss of generality, be taken to be diagonal. Second, we compute the state vector for one, two, and finally L encoding layers, and show that every amplitude is a sum of complex exponentials whose frequencies are sums of eigenvalues of the encoding Hamiltonian. Third, we evaluate the measured expectation value and show that the frequencies of the model output are *differences* of these eigenvalue sums. Finally, we collect terms of equal frequency to obtain the Fourier representation and illustrate it with Pauli-rotation encoding.

Conventions. We consider a scalar input $x \in \mathbb{R}$ and a circuit on N qubits, so that the Hilbert space has dimension $d = 2^N$. Computational basis states are indexed by $1, \ldots, d$, with the initial state $|0\rangle \equiv |0\rangle^{\otimes N}$ corresponding to the first basis vector. For a matrix W, W_{mn} denotes the entry in row m and column n. The symbol i is reserved for the imaginary unit; matrix and vector indices are denoted by m, n, j, and k.

Circuit model. The quantum circuits considered here share the general structure shown in Figure 11, in which data-independent blocks alternate with data-encoding gates. The overall unitary is therefore

$$U(x) = W^{(L+1)} S(x)\, W^{(L)} \cdots W^{(2)} S(x)\, W^{(1)}, \tag{4.9}$$

where each $W^{(l)}$ is a unitary that does not depend on x. These blocks may contain trainable parameterized gates, such as $R_X(\theta)$, as well as fixed gates, such as the Hadamard gate. The gate $S(x)$ encodes the classical input x into the quantum state and is repeated L times. We assume that $S(x)$ is generated by a Hermitian operator H, referred to as the *encoding Hamiltonian*:

$$S(x) = e^{-ixH}. \tag{4.10}$$

For example, encoding with the single-qubit rotation $R_X(x)$ corresponds to

$$H = \frac{1}{2} X = \frac{1}{2} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \tag{4.11}$$

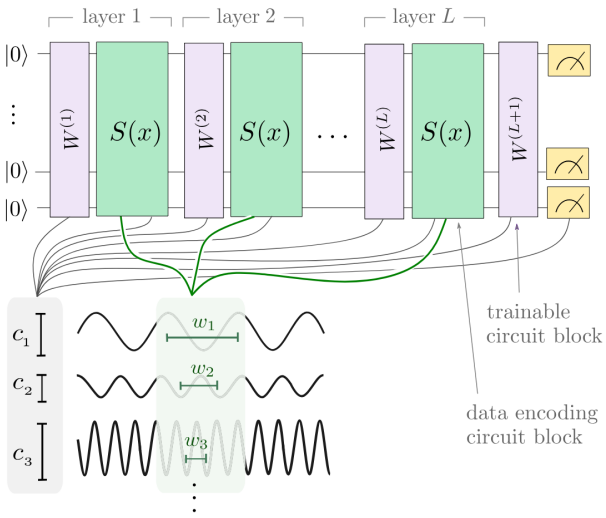


Figure 11: General structure of the quantum circuits considered in this section: data-independent blocks $W^{(l)}$ alternate with data-encoding gates $S(x)$.

Indeed, since $X^2 = I$, the Taylor series of the exponential separates into even powers, which are proportional to I, and odd powers, which are proportional to X:

$$\begin{aligned} e^{-i\frac{x}{2}X} &= \sum_{n=0}^{\infty} \frac{1}{n!} \left(-\frac{ix}{2}\right)^n X^n \\ &= \left[\sum_{k=0}^{\infty} \frac{(-1)^k}{(2k)!} \left(\frac{x}{2}\right)^{2k}\right] I - i \left[\sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)!} \left(\frac{x}{2}\right)^{2k+1}\right] X \\ &= \cos\left(\frac{x}{2}\right) I - i \sin\left(\frac{x}{2}\right) X = R_X(x). \end{aligned} \tag{4.12}$$

The output of the model is the expectation value of a Hermitian observable M (which need not be unitary) with respect to the final state:

$$f(x) = \langle \psi(x) | \, M \, | \psi(x) \rangle \,, \qquad |\psi(x)\rangle = U(x) \, |0\rangle \,. \tag{4.13}$$

The trainable parameters θ enter only through the blocks $W^{(l)}$; for brevity, we suppress the dependence on θ throughout.

Step 1: Diagonalizing the encoding gate. Because H is Hermitian, the spectral theorem guarantees the decomposition

$$H = V^\dagger \Sigma V, \qquad \Sigma = \mathrm{diag}(\lambda_1, \ldots, \lambda_d), \tag{4.14}$$

where V is unitary and the eigenvalues $\lambda_1, \ldots, \lambda_d$ are real. Since V is unitary, $(V^\dagger \Sigma V)^n = V^\dagger \Sigma^n V$ for every n, and the power-series definition of the matrix exponential gives

$$S(x) = V^\dagger e^{-ix\Sigma} V, \qquad e^{-ix\Sigma} = \mathrm{diag}\left(e^{-ix\lambda_1}, \ldots, e^{-ix\lambda_d}\right). \tag{4.15}$$

Substituting this expression into Eq. (4.9), every factor V or $V^\dagger$ is adjacent to a data-independent block:

$$U(x) = W^{(L+1)}V^\dagger e^{-ix\Sigma}V\,W^{(L)}V^\dagger e^{-ix\Sigma}V\cdots V\,W^{(2)}V^\dagger e^{-ix\Sigma}V\,W^{(1)}. \tag{4.16}$$

These factors can therefore be absorbed into the neighbouring blocks by defining

$$\widetilde{W}^{(1)} = VW^{(1)}, \qquad \widetilde{W}^{(l)} = VW^{(l)}V^\dagger \;\; (2 \le l \le L), \qquad \widetilde{W}^{(L+1)} = W^{(L+1)}V^\dagger, \tag{4.17}$$

which yields

$$U(x) = \widetilde{W}^{(L+1)}e^{-ix\Sigma}\,\widetilde{W}^{(L)}\cdots\widetilde{W}^{(2)}e^{-ix\Sigma}\,\widetilde{W}^{(1)}. \tag{4.18}$$

The new blocks $\widetilde{W}^{(l)}$ are still unitary and independent of x. Consequently, we may assume without loss of generality that the encoding gate is diagonal in the computational basis. To simplify notation, we drop the tildes and write from now on

$$S(x) = \mathrm{diag}\Big(e^{-ix\lambda_1},\ldots,e^{-ix\lambda_d}\Big). \tag{4.19}$$

Step 2: A single encoding layer ($L = 1$). We now compute the amplitudes of the final state, beginning with the simplest case,

$$U(x) = W^{(2)}S(x)\,W^{(1)}. \tag{4.20}$$

Because $|0\rangle$ is the first basis vector, the first block maps it to the first column of $W^{(1)}$:

$$|\phi\rangle = W^{(1)}|0\rangle = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_d \end{bmatrix}, \qquad a_j = W^{(1)}_{j1}. \tag{4.21}$$

Since $S(x)$ is diagonal, the encoding gate multiplies each component by a phase:

$$S(x)\,|\phi\rangle = \begin{bmatrix} a_1e^{-ix\lambda_1} \\ a_2e^{-ix\lambda_2} \\ \vdots \\ a_de^{-ix\lambda_d} \end{bmatrix}. \tag{4.22}$$

Applying the second block $W^{(2)}$, the m-th component of the final state is

$$\begin{aligned} \psi_m(x) &= W^{(2)}_{m1}a_1e^{-ix\lambda_1} + W^{(2)}_{m2}a_2e^{-ix\lambda_2} + \cdots + W^{(2)}_{md}a_de^{-ix\lambda_d} \\ &= \sum_{j=1}^{d} W^{(2)}_{mj}W^{(1)}_{j1}\,e^{-ix\lambda_j}. \end{aligned} \tag{4.23}$$

Each amplitude is thus a linear combination of the exponentials $e^{-ix\lambda_j}$, with coefficients that depend only on the data-independent blocks.

Step 3: Two encoding layers ($L = 2$). Next, consider

$$U(x) = W^{(3)} S(x)\, W^{(2)} S(x)\, W^{(1)}. \tag{4.24}$$

Let $\phi^{(l)}$ denote the state immediately after block $W^{(l)}$, and $\hat{\phi}^{(l)}$ the state after the encoding gate that follows it. The indices $j_1, j_2 \in \{1, \ldots, d\}$ are independent. After the first block,

$$\phi^{(1)}_{j_1} = W^{(1)}_{j_1 1}, \tag{4.25}$$

and after the first encoding gate,

$$\hat{\phi}^{(1)}_{j_1} = e^{-ix\lambda_{j_1}} W^{(1)}_{j_1 1}. \tag{4.26}$$

Applying the second block gives

$$\phi^{(2)}_{j_2} = \sum_{j_1=1}^{d} W^{(2)}_{j_2 j_1} e^{-ix\lambda_{j_1}} W^{(1)}_{j_1 1}. \tag{4.27}$$

The sum arises because the j_2-th row of $W^{(2)}$ combines all d components of the preceding state. The second encoding gate multiplies the j_2-th component by $e^{-ix\lambda_{j_2}}$. Since $e^{-ix\lambda_{j_2}} e^{-ix\lambda_{j_1}} = e^{-ix(\lambda_{j_1}+\lambda_{j_2})}$, we obtain

$$\hat{\phi}^{(2)}_{j_2} = e^{-ix\lambda_{j_2}} \phi^{(2)}_{j_2} = \sum_{j_1=1}^{d} W^{(2)}_{j_2 j_1} W^{(1)}_{j_1 1}\, e^{-ix(\lambda_{j_1}+\lambda_{j_2})}. \tag{4.28}$$

Finally, applying the third block yields

$$\psi_m(x) = \sum_{j_2=1}^{d} W^{(3)}_{m j_2} \hat{\phi}^{(2)}_{j_2} = \sum_{j_1=1}^{d} \sum_{j_2=1}^{d} W^{(3)}_{m j_2} W^{(2)}_{j_2 j_1} W^{(1)}_{j_1 1}\, e^{-ix(\lambda_{j_1}+\lambda_{j_2})}. \tag{4.29}$$

Each term factorizes into two parts:

1. an input-independent coefficient, $W^{(3)}_{m j_2} W^{(2)}_{j_2 j_1} W^{(1)}_{j_1 1}$, determined by the data-independent blocks; and
2. an input-dependent phase, $e^{-ix(\lambda_{j_1}+\lambda_{j_2})}$, whose frequency is a sum of eigenvalues of the encoding Hamiltonian, one from each encoding layer.

Step 4: General number of encoding layers. The same pattern persists for any number of layers: each additional encoding gate contributes one phase factor, and each additional block introduces one summation index. Formally, the result follows by induction on L. If the components after block $W^{(l)}$ are sums over $j_1, \ldots, j_{l-1}$ of products of block entries multiplied by $e^{-ix(\lambda_{j_1}+\cdots+\lambda_{j_{l-1}})}$, then the subsequent encoding gate multiplies component j_l by $e^{-ix\lambda_{j_l}}$, and block $W^{(l+1)}$ introduces a sum over j_l; this reproduces the same form with l replaced by $l+1$. For L encoding layers we therefore have

$$\psi_m(x) = \sum_{j_1,\ldots,j_L=1}^{d} A_{m,j_1,\ldots,j_L}\, e^{-ix(\lambda_{j_1}+\cdots+\lambda_{j_L})}, \tag{4.30}$$

where the coefficients are products of entries of the data-independent blocks:

$$A_{m,j_1,\ldots,j_L} = W^{(L+1)}_{mj_L} W^{(L)}_{j_L j_{L-1}} \cdots W^{(2)}_{j_2 j_1} W^{(1)}_{j_1 1}. \tag{4.31}$$

To shorten the notation, we collect the indices into a multi-index and define the corresponding eigenvalue sum:

$$\mathbf{j} = (j_1, \ldots, j_L) \in [d]^L, \qquad \Lambda_{\mathbf{j}} = \lambda_{j_1} + \lambda_{j_2} + \cdots + \lambda_{j_L}, \tag{4.32}$$

where $[d] = \{1, \ldots, d\}$, so that $[d]^L$ is the set of all d^L index tuples of length L. The amplitudes then take the compact form

$$\psi_m(x) = \sum_{\mathbf{j} \in [d]^L} A_{m,\mathbf{j}}\, e^{-ix\Lambda_{\mathbf{j}}}. \tag{4.33}$$

In words, regardless of the depth of the circuit, every amplitude is a sum of terms of the form (circuit coefficient) $\times\, e^{-ix\cdot(\text{eigenvalue sum})}$. The coefficients depend on the trainable blocks, whereas the eigenvalue sums depend only on the encoding Hamiltonian and on L.

For example, with $d = 2$ and $L = 2$, the multi-index ranges over

$$\mathbf{j} \in \{(1,1), (1,2), (2,1), (2,2)\}, \tag{4.34}$$

and Eq. (4.33) contains four terms:

$$\psi_m(x) = A_{m,(1,1)} e^{-2ix\lambda_1} + A_{m,(1,2)} e^{-ix(\lambda_1+\lambda_2)} + A_{m,(2,1)} e^{-ix(\lambda_2+\lambda_1)} + A_{m,(2,2)} e^{-2ix\lambda_2}. \tag{4.35}$$

The multi-indices $(1,2)$ and $(2,1)$ yield the same eigenvalue sum, so the corresponding terms share a frequency. Such coincidences are common and will be used in Step 6 to combine terms.

Step 5: Measurement. The amplitudes contain frequencies $\Lambda_{\mathbf{j}}$, whereas the frequencies of the model output turn out to be differences $\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}}$. This change originates in the measurement: the expectation value in Eq. (4.13) is quadratic in the state and involves both the amplitudes and their complex conjugates. Writing

$$|\psi(x)\rangle = \begin{bmatrix} \psi_1(x) \\ \vdots \\ \psi_d(x) \end{bmatrix}, \qquad \langle\psi(x)| = \begin{bmatrix} \psi_1^*(x) & \cdots & \psi_d^*(x) \end{bmatrix}, \tag{4.36}$$

the expectation value becomes

$$f(x) = \sum_{m=1}^{d} \sum_{n=1}^{d} \psi_m^*(x)\, M_{mn}\, \psi_n(x). \tag{4.37}$$

For a single qubit ($d = 2$), for instance,

$$\begin{aligned} f(x) &= \begin{bmatrix} \psi_1^* & \psi_2^* \end{bmatrix} \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix} \begin{bmatrix} \psi_1 \\ \psi_2 \end{bmatrix} \\ &= \psi_1^* M_{11} \psi_1 + \psi_1^* M_{12} \psi_2 + \psi_2^* M_{21} \psi_1 + \psi_2^* M_{22} \psi_2. \end{aligned} \tag{4.38}$$

The off-diagonal entries of M couple different amplitudes through cross terms. Even when M is diagonal (e.g., $M = Z$), the output contains the products $\psi_m^* \psi_m$ and therefore remains quadratic in the amplitudes.

To evaluate Eq. (4.37), we require the complex conjugate of Eq. (4.33). Because the eigenvalue sums $\Lambda_{\mathbf{k}}$ are real,

$$\psi_m^*(x) = \sum_{\mathbf{k} \in [d]^L} A_{m,\mathbf{k}}^* \, e^{ix\Lambda_{\mathbf{k}}}. \tag{4.39}$$

Conjugation has two effects: each coefficient is replaced by its complex conjugate, and the sign of the exponent changes from negative to positive. We also rename the summation multi-index to $\mathbf{k}$; it ranges over the same set $[d]^L$ as $\mathbf{j}$, but the two sums are independent. Substituting both expansions into Eq. (4.37) gives

$$\begin{aligned} f(x) &= \sum_{m,n} \left(\sum_{\mathbf{k}} A_{m,\mathbf{k}}^* \, e^{ix\Lambda_{\mathbf{k}}} \right) M_{mn} \left(\sum_{\mathbf{j}} A_{n,\mathbf{j}} \, e^{-ix\Lambda_{\mathbf{j}}} \right) \\ &= \sum_{m,n} \sum_{\mathbf{k},\mathbf{j}} A_{m,\mathbf{k}}^* \, M_{mn} \, A_{n,\mathbf{j}} \, e^{ix\Lambda_{\mathbf{k}}} e^{-ix\Lambda_{\mathbf{j}}}. \end{aligned} \tag{4.40}$$

Using $e^{ix\Lambda_{\mathbf{k}}} e^{-ix\Lambda_{\mathbf{j}}} = e^{ix(\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}})}$ and exchanging the order of summation, we obtain

$$f(x) = \sum_{\mathbf{k},\mathbf{j} \in [d]^L} \left(\sum_{m,n} A_{m,\mathbf{k}}^* \, M_{mn} \, A_{n,\mathbf{j}} \right) e^{ix(\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}})}. \tag{4.41}$$

Defining the input-independent coefficients

$$a_{\mathbf{k},\mathbf{j}} = \sum_{m,n} A_{m,\mathbf{k}}^* \, M_{mn} \, A_{n,\mathbf{j}}, \tag{4.42}$$

the model output becomes

$$f(x) = \sum_{\mathbf{k},\mathbf{j} \in [d]^L} a_{\mathbf{k},\mathbf{j}} \, e^{ix(\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}})}. \tag{4.43}$$

Thus the output is a sum of terms of the form (coefficient) $\times \, e^{ix \cdot (\text{difference of eigenvalue sums})}$.

Step 6: Frequency spectrum and Fourier coefficients. Comparing each term in Eq. (4.43) with the Fourier basis function $e^{i\omega x}$, we identify its frequency as

$$\omega = \Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} = \sum_{l=1}^{L} \lambda_{k_l} - \sum_{l=1}^{L} \lambda_{j_l}. \tag{4.44}$$

The set of frequencies accessible to the model, its *frequency spectrum*, is therefore

$$\Omega = \left\{ \Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} \ : \ \mathbf{k}, \mathbf{j} \in [d]^L \right\}. \tag{4.45}$$

To summarize the notation: λ denotes an eigenvalue of the encoding Hamiltonian, Λ a sum of eigenvalues across the L encoding layers, and ω a difference of two such sums.

As anticipated in Step 4, distinct pairs of multi-indices may produce the same frequency. If, for example, $\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} = \Lambda_{\mathbf{k}'} - \Lambda_{\mathbf{j}'} = \omega$ for two different pairs $(\mathbf{k}, \mathbf{j})$ and $(\mathbf{k}', \mathbf{j}')$, the corresponding terms combine as

$$a_{\mathbf{k},\mathbf{j}}\, e^{i\omega x} + a_{\mathbf{k}',\mathbf{j}'}\, e^{i\omega x} = (a_{\mathbf{k},\mathbf{j}} + a_{\mathbf{k}',\mathbf{j}'})\, e^{i\omega x}. \tag{4.46}$$

Collecting all pairs that share a frequency, we define the Fourier coefficient associated with $\omega \in \Omega$ as

$$c_\omega = \sum_{\substack{\mathbf{k},\mathbf{j} \in [d]^L \\ \Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} = \omega}} a_{\mathbf{k},\mathbf{j}}, \tag{4.47}$$

and arrive at the Fourier representation of the QNN:

$$\boxed{f(x) = \sum_{\omega \in \Omega} c_\omega\, e^{i\omega x}} \tag{4.48}$$

Since Ω contains at most d^{2L} elements, this is a finite sum. When the eigenvalue differences are commensurate, as for the Pauli encodings discussed below, all frequencies are integer multiples of a fundamental frequency and Eq. (4.48) is a truncated Fourier series in the usual sense; in general, it is a finite sum of complex exponentials.

The representation has two further properties. First, the spectrum is symmetric and contains zero: if $\omega = \Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} \in \Omega$, then $-\omega = \Lambda_{\mathbf{j}} - \Lambda_{\mathbf{k}} \in \Omega$, and $\mathbf{k} = \mathbf{j}$ gives $\omega = 0$. Second, because M is Hermitian ($M^*_{mn} = M_{nm}$), Eq. (4.42) implies

$$a^*_{\mathbf{k},\mathbf{j}} = \sum_{m,n} A_{m,\mathbf{k}}\, M^*_{mn}\, A^*_{n,\mathbf{j}} = \sum_{m,n} A^*_{n,\mathbf{j}}\, M_{nm}\, A_{m,\mathbf{k}} = a_{\mathbf{j},\mathbf{k}}, \tag{4.49}$$

and summing over all pairs with frequency ω gives $c_{-\omega} = c^*_\omega$. Consequently, $f(x)$ is real-valued, as an expectation value of a Hermitian observable must be, and can equivalently be written as

$$f(x) = c_0 + \sum_{\omega \in \Omega,\, \omega > 0} 2\,|c_\omega| \cos(\omega x + \arg c_\omega)\,. \tag{4.50}$$

Roles of the encoding and trainable blocks. The blocks $W^{(l)}$ do not appear in the frequency spectrum (4.45), which is determined entirely by the eigenvalues of the encoding Hamiltonian and the number of encoding layers L. This is because the blocks are independent of x: they enter only through the coefficients $A_{m,\mathbf{j}}$ in Eq. (4.31), and hence only through $a_{\mathbf{k},\mathbf{j}}$ and c_ω, while leaving the exponential factors $e^{i\omega x}$ unchanged. Training therefore adjusts the amplitudes and phases of a fixed set of frequencies but cannot introduce frequencies outside Ω. The extent to which the coefficients themselves can be controlled depends on the expressivity of the trainable blocks. If different encoding Hamiltonians $H^{(l)}$ are used in different layers, the same derivation applies with $\Lambda_{\mathbf{j}} = \sum_l \lambda^{(l)}_{j_l}$, where $\lambda^{(l)}_j$ are the eigenvalues of $H^{(l)}$.

Example: Pauli-rotation encoding. For single-qubit encoding with $R_X(x)$, we have $H = \frac{1}{2}X$ with eigenvalues $\pm\frac{1}{2}$. Each eigenvalue sum $\Lambda_{\mathbf{j}}$ is a sum of L terms equal to $\pm\frac{1}{2}$ and

therefore takes values in $\{-\frac{L}{2}, -\frac{L}{2}+1, \ldots, \frac{L}{2}\}$. Their differences are the integers from $-L$ to L, so that

$$\Omega = \{-L, \ldots, -1, 0, 1, \ldots, L\}, \qquad f(x) = \sum_{\omega=-L}^{L} c_\omega \, e^{i\omega x}. \tag{4.51}$$

A model that encodes x L times through a single-qubit Pauli rotation is thus a truncated Fourier series of degree L. In particular, with a single encoding layer ($L = 1$), the model reduces to $f(x) = c_0 + 2|c_1| \cos(x + \arg c_1)$, a single sinusoid, irrespective of how expressive the trainable blocks are. Accessing higher frequencies requires encoding the input repeatedly, either sequentially (data re-uploading, larger L) or in parallel; for instance, applying $R_X(x)$ simultaneously to N qubits gives $H = \frac{1}{2}\sum_{q=1}^{N} X_q$, whose eigenvalues range over $\{-\frac{N}{2}, \ldots, \frac{N}{2}\}$, so a single such layer already yields the spectrum $\{-N, \ldots, N\}$.

4.4 Expressivity and Frequency Expansion

5 Applications

References

[1] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521:436–444, 2015.

[2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[3] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009.

[4] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010.

[5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[6] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[7] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989.

[8] Moshe Leshno, Vladimir Ya. Lin, Allan Pinkus, and Shimon Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867, 1993.

[9] Matus Telgarsky. Benefits of depth in neural networks. In *29th Annual Conference on Learning Theory*, volume 49 of *Proceedings of Machine Learning Research*, pages 1517–1539. PMLR, 2016.

[10] Kosuke Mitarai, Makoto Negoro, Masahiro Kitagawa, and Keisuke Fujii. Quantum circuit learning. *Physical Review A*, 98(3):032309, 2018.

[11] Marcello Benedetti, Erika Lloyd, Stefan Sack, and Mattia Fiorentini. Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, 4(4):043001, 2019.

[12] Adrián Pérez-Salinas, Alba Cervera-Lierta, Elies Gil-Fuster, and José I. Latorre. Data re-uploading for a universal quantum classifier. *Quantum*, 4:226, 2020.

[13] Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. Effect of data encoding on the expressive power of variational quantum-machine-learning models. *Physical Review A*, 103(3):032430, 2021.

[14] Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A. Smolin, and Harald Weinfurter. Elementary gates for quantum computation. *Physical Review A*, 52(5):3457–3467, 1995.

[15] Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[16] Elias M. Stein and Rami Shakarchi. *Fourier Analysis: An Introduction*, volume 1 of *Princeton Lectures in Analysis*. Princeton University Press, 2003.